Table of Contents

1 Automata: The Methods and the Madness **1**

1.1 Why Study Automata Theory? 2
1.1.1 Introduction to Finite Automata 2
1.1.2 Structural Representations 4
1.1.3 Automata and Complexity 5
1.2 Introduction to Formal Proof 5
1.2.1 Deductive Proofs 6
1.2.2 Reduction to Definitions 8
1.2.3 Other Theorem Forms 10
1.2.4 Theorems That Appear Not to Be If-Then Statements . . 13
1.3 Additional Forms of Proof 13
1.3.1 Proving Equivalences About Sets 14
1.3.2 The Contrapositive 14
1.3.3 Proof by Contradiction 16
1.3.4 Counterexamples 17
1.4 Inductive Proofs 19
1.4.1 Inductions on Integers 19
1.4.2 More General Forms of Integer Inductions 22
1.4.3 Structural Inductions 23
1.4.4 Mutual Inductions 26
1.5 The Central Concepts of Automata Theory 28
1.5.1 Alphabets 28
1.5.2 Strings 29
1.5.3 Languages 30
1.5.4 Problems 31
1.6 Summary of Chapter 1 33
1.7 Gradiance Problems for Chapter 1 35
1.8 References for Chapter 1 36

2 Finite Automata **37**

2.1 An Informal Picture of Finite Automata 38
2.1.1 The Ground Rules 38
2.1.2 The Protocol 39

Chapter 2

Finite Automata

This chapter introduces the class of languages known as "regular languages." These languages are exactly the ones that can be described by finite automata, which we sampled briefly in Section 1.1.1. After an extended example that will provide motivation for the study to follow, we define finite automata formally.

As was mentioned earlier, a finite automaton has a set of states, and its "control" moves from state to state in response to external "inputs." One of the crucial distinctions among classes of finite automata is whether that control is "deterministic," meaning that the automaton cannot be in more than one state at any one time, or "nondeterministic," meaning that it may be in several states at once. We shall discover that adding nondeterminism does not let us define any language that cannot be defined by a deterministic finite automaton, but there can be substantial efficiency in describing an application using a nondeterministic automaton. In effect, nondeterminism allows us to "program" solutions to problems using a higher-level language. The nondeterministic finite automaton is then "compiled," by an algorithm we shall learn in this chapter, into a deterministic automaton that can be "executed" on a conventional computer.

We conclude the chapter with a study of an extended nondeterministic automaton that has the additional choice of making a transition from one state to another spontaneously, i.e., on the empty string as "input." These automata also accept nothing but the regular languages. However, we shall find them quite important in Chapter 3, when we study regular expressions and their equivalence to automata.

The study of the regular languages continues in Chapter 3. There, we introduce another important way to describe regular languages: the algebraic notation known as regular expressions. After discussing regular expressions, and showing their equivalence to finite automata, we use both automata and regular expressions as tools in Chapter 4 to show certain important properties of the regular languages. Examples of such properties are the "closure" properties, which allow us to claim that one language is regular because one or more

other languages are known to be regular, and "decision" properties. The latter are algorithms to answer questions about automata or regular expressions, e.g., whether two automata or expressions represent the same language.

2.1 An Informal Picture of Finite Automata

In this section, we shall study an extended example of a real-world problem whose solution uses finite automata in an important role. We investigate protocols that support "electronic money" — files that a customer can use to pay for goods on the internet, and that the seller can receive with assurance that the "money" is real. The seller must know that the file has not been forged, nor has it been copied and sent to the seller, while the customer retains a copy of the same file to spend again.

The nonforgeability of the file is something that must be assured by a bank and by a cryptography policy. That is, a third player, the bank, must issue and encrypt the "money" files, so that forgery is not a problem. However, the bank has a second important job: it must keep a database of all the valid money that it has issued, so that it can verify to a store that the file it has received represents real money and can be credited to the store's account. We shall not address the cryptographic aspects of the problem, nor shall we worry about how the bank can store and retrieve what could be billions of "electronic dollar bills." These problems are not likely to represent long-term impediments to the concept of electronic money, and examples of its small-scale use have existed since the late 1990's.

However, in order to use electronic money, protocols need to be devised to allow the manipulation of the money in a variety of ways that the users want. Because monetary systems always invite fraud, we must verify whatever policy we adopt regarding how money is used. That is, we need to prove the only things that can happen are things we intend to happen — things that do not allow an unscrupulous user to steal from others or to "manufacture" money. In the balance of this section, we shall introduce a very simple example of a (bad) electronic-money protocol, model it with finite automata, and show how constructions on automata can be used to verify protocols (or, in this case, to discover that the protocol has a bug).

2.1.1 The Ground Rules

There are three participants: the customer, the store, and the bank. We assume for simplicity that there is only one "money" file in existence. The customer may decide to transfer this money file to the store, which will then redeem the file from the bank (i.e., get the bank to issue a new money file belonging to the store rather than the customer) and ship goods to the customer. In addition, the customer has the option to cancel the file. That is, the customer may ask the bank to place the money back in the customer's account, making the money

no longer spendable. Interaction among the three participants is thus limited to five events:

1. The customer may decide to *pay*. That is, the customer sends the money to the store.
2. The customer may decide to *cancel*. The money is sent to the bank with a message that the value of the money is to be added to the customer's bank account.
3. The store may *ship* goods to the customer.
4. The store may *redeem* the money. That is, the money is sent to the bank with a request that its value be given to the store.
5. The bank may *transfer* the money by creating a new, suitably encrypted money file and sending it to the store.

2.1.2 The Protocol

The three participants must design their behaviors carefully, or the wrong things may happen. In our example, we make the reasonable assumption that the customer cannot be relied upon to act responsibly. In particular, the customer may try to copy the money file, use it to pay several times, or both pay and cancel the money, thus getting the goods "for free."

The bank must behave responsibly, or it cannot be a bank. In particular, it must make sure that two stores cannot both redeem the same money file, and it must not allow money to be both canceled and redeemed. The store should be careful as well. In particular, it should not ship goods until it is sure it has been given valid money for the goods.

Protocols of this type can be represented as finite automata. Each state represents a situation that one of the participants could be in. That is, the state "remembers" that certain important events have happened and that others have not yet happened. Transitions between states occur when one of the five events described above occur. We shall think of these events as "external" to the automata representing the three participants, even though each participant is responsible for initiating one or more of the events. It turns out that what is important about the problem is what sequences of events can happen, not who is allowed to initiate them.

Figure 2.1 represents the three participants by automata. In that diagram, we show only the events that affect a participant. For example, the action *pay* affects only the customer and store. The bank does not know that the money has been sent by the customer to the store; it discovers that fact only when the store executes the action *redeem*.

Let us examine first the automaton (c) for the bank. The start state is state 1; it represents the situation where the bank has issued the money file in question but has not been requested either to redeem it or to cancel it. If a

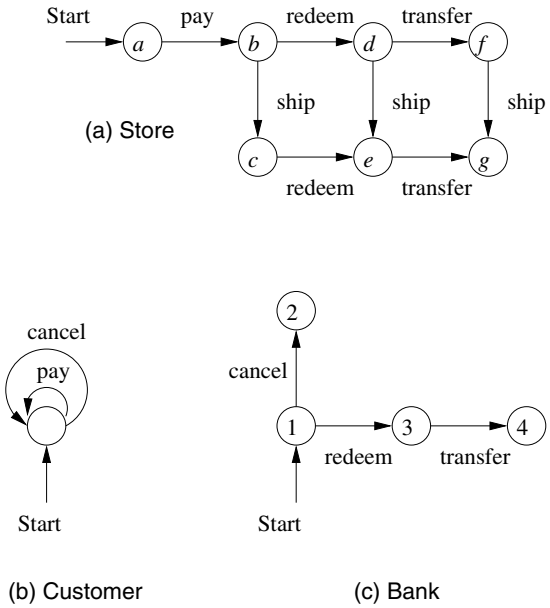


Figure 2.1: Finite automata representing a customer, a store, and a bank

cancel request is sent to the bank by the customer, then the bank restores the money to the customer's account and enters state 2. The latter state represents the situation where the money has been cancelled. The bank, being responsible, will not leave state 2 once it is entered, since the bank must not allow the same money to be cancelled again or spent by the customer.[1]

Alternatively, when in state 1 the bank may receive a *redeem* request from the store. If so, it goes to state 3, and shortly sends the store a *transfer* message, with a new money file that now belongs to the store. After sending the transfer message, the bank goes to state 4. In that state, it will neither accept *cancel* or *redeem* requests nor will it perform any other actions regarding this particular money file.

Now, let us consider Fig. 2.1(a), the automaton representing the actions of the store. While the bank always does the right thing, the store's system has some defects. Imagine that the shipping and financial operations are done by separate processes, so there is the opportunity for the *ship* action to be done either before, after, or during the redemption of the electronic money. That policy allows the store to get into a situation where it has already shipped the goods and then finds out the money was bogus.

The store starts out in state a. When the customer orders the goods by

[1] You should remember that this entire discussion is about one single money file. The bank will in fact be running the same protocol with a large number of electronic pieces of money, but the workings of the protocol are the same for each of them, so we can discuss the problem as if there were only one piece of electronic money in existence.

performing the *pay* action, the store enters state b. In this state, the store begins both the shipping and redemption processes. If the goods are shipped first, then the store enters state c, where it must still redeem the money from the bank and receive the *transfer* of an equivalent money file from the bank. Alternatively, the store may send the *redeem* message first, entering state d. From state d, the store might next ship, entering state e, or it might next receive the transfer of money from the bank, entering state f. From state f, we expect that the store will eventually ship, putting the store in state g, where the transaction is complete and nothing more will happen. In state e, the store is waiting for the *transfer* from the bank. Unfortunately, the goods have already been shipped, and if the *transfer* never occurs, the store is out of luck.

Last, observe the automaton for the customer, Fig. 2.1(b). This automaton has only one state, reflecting the fact that the customer "can do anything." The customer can perform the *pay* and *cancel* actions any number of times, in any order, and stays in the lone state after each action.

2.1.3 Enabling the Automata to Ignore Actions

While the three automata of Fig. 2.1 reflect the behaviors of the three participants independently, there are certain transitions that are missing. For example, the store is not affected by a *cancel* message, so if the *cancel* action is performed by the customer, the store should remain in whatever state it is in. However, in the formal definition of a finite automaton, which we shall study in Section 2.2, whenever an input X is received by an automaton, the automaton must follow an arc labeled X from the state it is in to some new state. Thus, the automaton for the store needs an additional arc from each state to itself, labeled *cancel*. Then, whenever the *cancel* action is executed, the store automaton can make a "transition" on that input, with the effect that it stays in the same state it was in. Without these additional arcs, whenever the *cancel* action was executed the store automaton would "die"; that is, the automaton would be in no state at all, and further actions by that automaton would be impossible.

Another potential problem is that one of the participants may, intentionally or erroneously, send an unexpected message, and we do not want this action to cause one of the automata to die. For instance, suppose the customer decided to execute the *pay* action a second time, while the store was in state e. Since that state has no arc out with label *pay*, the store's automaton would die before it could receive the transfer from the bank. In summary, we must add to the automata of Fig. 2.1 loops on certain states, with labels for all those actions that must be ignored when in that state; the complete automata are shown in Fig. 2.2. To save space, we combine the labels onto one arc, rather than showing several arcs with the same heads and tails but different labels. The two kinds of actions that must be ignored are:

1. *Actions that are irrelevant to the participant involved.* As we saw, the only irrelevant action for the store is *cancel*, so each of its seven states

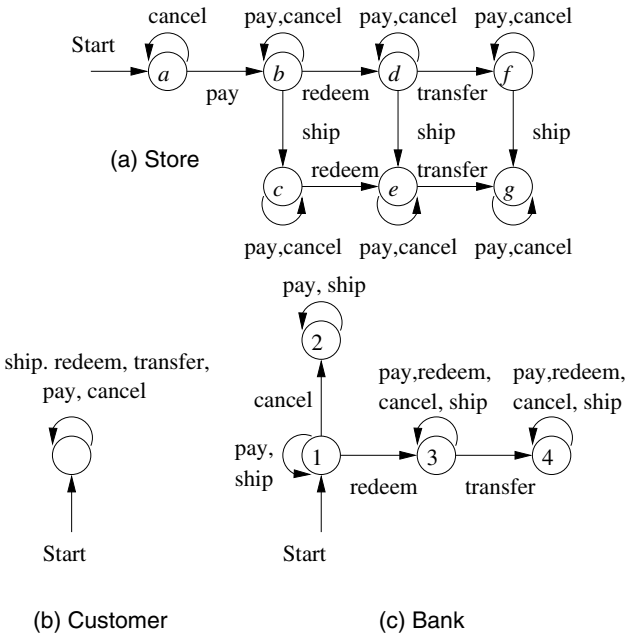


Figure 2.2: The complete sets of transitions for the three automata

has a loop labeled *cancel*. For the bank, both *pay* and *ship* are irrelevant, so we have put at each of the bank's states an arc labeled *pay, ship*. For the customer, *ship*, *redeem* and *transfer* are all irrelevant, so we add arcs with these labels. In effect, it stays in its one state on any sequence of inputs, so the customer automaton has no effect on the operation of the overall system. Of course, the customer is still a participant, since it is the customer who initiates the *pay* and *cancel* actions. However, as we mentioned, the matter of who initiates actions has nothing to do with the behavior of the automata.

2. *Actions that must not be allowed to kill an automaton.* As mentioned, we must not allow the customer to kill the store's automaton by executing *pay* again, so we have added loops with label *pay* to all but state a (where the *pay* action is expected and relevant). We have also added loops with labels *cancel* to states 3 and 4 of the bank, in order to prevent the customer from killing the bank's automaton by trying to cancel money that has already been redeemed. The bank properly ignores such a request. Likewise, states 3 and 4 have loops on *redeem*. The store should not try to redeem the same money twice, but if it does, the bank properly ignores the second request.

2.1.4 The Entire System as an Automaton

While we now have models for how the three participants behave, we do not yet have a representation for the interaction of the three participants. As mentioned, because the customer has no constraints on behavior, that automaton has only one state, and any sequence of events lets it stay in that state; i.e., it is not possible for the system as a whole to "die" because the customer automaton has no response to an action. However, both the store and bank behave in a complex way, and it is not immediately obvious in what combinations of states these two automata can be.

The normal way to explore the interaction of automata such as these is to construct the *product* automaton. That automaton's states represent a pair of states, one from the store and one from the bank. For instance, the state $(3, d)$ of the product automaton represents the situation where the bank is in state 3, and the store is in state d. Since the bank has four states and the store has seven, the product automaton has $4 \times 7 = 28$ states.

We show the product automaton in Fig. 2.3. For clarity, we have arranged the 28 states in an array. The row corresponds to the state of the bank and the column to the state of the store. To save space, we have also abbreviated the labels on the arcs, with P, S, C, R, and T standing for pay, ship, cancel, redeem, and transfer, respectively.

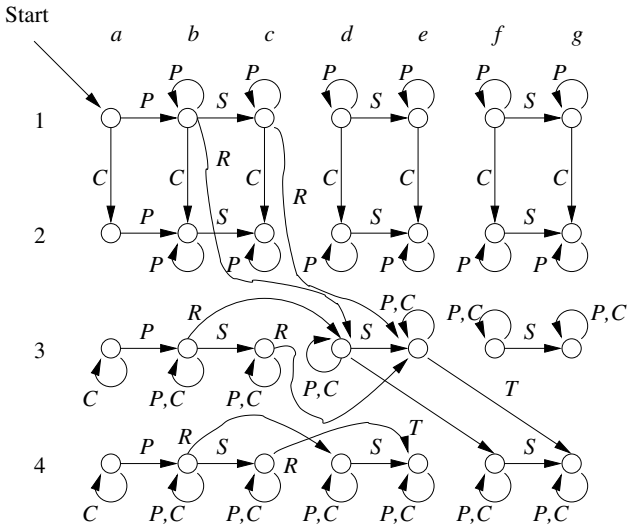


Figure 2.3: The product automaton for the store and bank

To construct the arcs of the product automaton, we need to run the bank and store automata "in parallel." Each of the two components of the product automaton independently makes transitions on the various inputs. However, it is important to notice that if an input action is received, and one of the two

automata has no state to go to on that input, then the product automaton "dies"; it has no state to go to.

To make this rule for state transitions precise, suppose the product automaton is in state (i, x). That state corresponds to the situation where the bank is in state i and the store in state x. Let Z be one of the input actions. We look at the automaton for the bank, and see whether there is a transition out of state i with label Z. Suppose there is, and it leads to state j (which might be the same as i if the bank loops on input Z). Then, we look at the store and see if there is an arc labeled Z leading to some state y. If both j and y exist, then the product automaton has an arc from state (i, x) to state (j, y), labeled Z. If either of states j or y do not exist (because the bank or store has no arc out of i or x, respectively, for input Z), then there is no arc out of (i, x) labeled Z.

We can now see how the arcs of Fig. 2.3 were selected. For instance, on input *pay*, the store goes from state a to b, but stays put if it is in any other state besides a. The bank stays in whatever state it is in when the input is *pay*, because that action is irrelevant to the bank. This observation explains the four arcs labeled P at the left ends of the four rows in Fig. 2.3, and the loops labeled P on other states.

For another example of how the arcs are selected, consider the input *redeem*. If the bank receives a *redeem* message when in state 1, it goes to state 3. If in states 3 or 4, it stays there, while in state 2 the bank automaton dies; i.e., it has nowhere to go. The store, on the other hand, can make transitions from state b to d or from c to e when the *redeem* input is received. In Fig. 2.3, we see six arcs labeled *redeem*, corresponding to the six combinations of three bank states and two store states that have outward-bound arcs labeled R. For example, in state $(1, b)$, the arc labeled R takes the automaton to state $(3, d)$, since *redeem* takes the bank from state 1 to 3 and the store from b to d. As another example, there is an arc labeled R from $(4, c)$ to $(4, e)$, since *redeem* takes the bank from state 4 back to state 4, while it takes the store from state c to state e.

2.1.5 Using the Product Automaton to Validate the Protocol

Figure 2.3 tells us some interesting things. For instance, of the 28 states, only ten of them can be reached from the start state, which is $(1, a)$ — the combination of the start states of the bank and store automata. Notice that states like $(2, e)$ and $(4, d)$ are not *accessible*, that is, there is no path to them from the start state. Inaccessible states need not be included in the automaton, and we did so in this example just to be systematic.

However, the real purpose of analyzing a protocol such as this one using automata is to ask and answer questions that mean "can the following type of error occur?" In the example at hand, we might ask whether it is possible that the store can ship goods and never get paid. That is, can the product automaton get into a state in which the store has shipped (that is, the state is

in column c, e, or g), and yet no transition on input T was ever made or will be made?

For instance, in state $(3, e)$, the goods have shipped, but there will eventually be a transition on input T to state $(4, g)$. In terms of what the bank is doing, once it has gotten to state 3, it has received the *redeem* request and processed it. That means it must have been in state 1 before receiving the *redeem* and therefore the *cancel* message had not been received and will be ignored if received in the future. Thus, the bank will eventually perform the transfer of money to the store.

However, state $(2, c)$ is a problem. The state is accessible, but the only arc out leads back to that state. This state corresponds to a situation where the bank received a *cancel* message before a *redeem* message. However, the store received a *pay* message; i.e., the customer was being duplicitous and has both spent and canceled the same money. The store foolishly shipped before trying to redeem the money, and when the store does execute the *redeem* action, the bank will not even acknowledge the message, because it is in state 2, where it has canceled the money and will not process a *redeem* request.

2.2 Deterministic Finite Automata

Now it is time to present the formal notion of a finite automaton, so that we may start to make precise some of the informal arguments and descriptions that we saw in Sections 1.1.1 and 2.1. We begin by introducing the formalism of a deterministic finite automaton, one that is in a single state after reading any sequence of inputs. The term "deterministic" refers to the fact that on each input there is one and only one state to which the automaton can transition from its current state. In contrast, "nondeterministic" finite automata, the subject of Section 2.3, can be in several states at once. The term "finite automaton" will refer to the deterministic variety, although we shall use "deterministic" or the abbreviation *DFA* normally, to remind the reader of which kind of automaton we are talking about.

2.2.1 Definition of a Deterministic Finite Automaton

A *deterministic finite automaton* consists of:

1. A finite set of *states*, often denoted Q.

2. A finite set of *input symbols*, often denoted Σ.

3. A *transition function* that takes as arguments a state and an input symbol and returns a state. The transition function will commonly be denoted δ. In our informal graph representation of automata, δ was represented by arcs between states and the labels on the arcs. If q is a state, and a is an

input symbol, then $\delta(q, a)$ is that state p such that there is an arc labeled a from q to p.[2]

4. A *start state*, one of the states in Q.

5. A set of *final* or *accepting* states F. The set F is a subset of Q.

A deterministic finite automaton will often be referred to by its acronym: *DFA*. The most succinct representation of a DFA is a listing of the five components above. In proofs we often talk about a DFA in "five-tuple" notation:

$$A = (Q, \Sigma, \delta, q_0, F)$$

where A is the name of the DFA, Q is its set of states, Σ its input symbols, δ its transition function, q_0 its start state, and F its set of accepting states.

2.2.2 How a DFA Processes Strings

The first thing we need to understand about a DFA is how the DFA decides whether or not to "accept" a sequence of input symbols. The "language" of the DFA is the set of all strings that the DFA accepts. Suppose $a_1 a_2 \cdots a_n$ is a sequence of input symbols. We start out with the DFA in its start state, q_0. We consult the transition function δ, say $\delta(q_0, a_1) = q_1$ to find the state that the DFA A enters after processing the first input symbol a_1. We process the next input symbol, a_2, by evaluating $\delta(q_1, a_2)$; let us suppose this state is q_2. We continue in this manner, finding states $q_3, q_4, \ldots, q_n$ such that $\delta(q_{i-1}, a_i) = q_i$ for each i. If q_n is a member of F, then the input $a_1 a_2 \cdots a_n$ is accepted, and if not then it is "rejected."

Example 2.1: Let us formally specify a DFA that accepts all and only the strings of 0's and 1's that have the sequence 01 somewhere in the string. We can write this language L as:

$$\{w \mid w \text{ is of the form } x01y \text{ for some strings } x \text{ and } y \text{ consisting of 0's and 1's only}\}$$

Another equivalent description, using parameters x and y to the left of the vertical bar, is:

$$\{x01y \mid x \text{ and } y \text{ are any strings of 0's and 1's}\}$$

Examples of strings in the language include 01, 11010, and 100011. Examples of strings *not* in the language include ϵ, 0, and 111000.

What do we know about an automaton that can accept this language L? First, its input alphabet is $\Sigma = \{0, 1\}$. It has some set of states, Q, of which one, say q_0, is the start state. This automaton has to remember the important facts about what inputs it has seen so far. To decide whether 01 is a substring of the input, A needs to remember:

[2] More accurately, the graph is a picture of some transition function δ, and the arcs of the graph are constructed to reflect the transitions specified by δ.

1. Has it already seen 01? If so, then it accepts every sequence of further inputs; i.e., it will only be in accepting states from now on.

2. Has it never seen 01, but its most recent input was 0, so if it now sees a 1, it will have seen 01 and can accept everything it sees from here on?

3. Has it never seen 01, but its last input was either nonexistent (it just started) or it last saw a 1? In this case, A cannot accept until it first sees a 0 and then sees a 1 immediately after.

These three conditions can each be represented by a state. Condition (3) is represented by the start state, q_0. Surely, when just starting, we need to see a 0 and then a 1. But if in state q_0 we next see a 1, then we are no closer to seeing 01, and so we must stay in state q_0. That is, $\delta(q_0, 1) = q_0$.

However, if we are in state q_0 and we next see a 0, we are in condition (2). That is, we have never seen 01, but we have our 0. Thus, let us use q_2 to represent condition (2). Our transition from q_0 on input 0 is $\delta(q_0, 0) = q_2$.

Now, let us consider the transitions from state q_2. If we see a 0, we are no better off than we were, but no worse either. We have not seen 01, but 0 was the last symbol, so we are still waiting for a 1. State q_2 describes this situation perfectly, so we want $\delta(q_2, 0) = q_2$. If we are in state q_2 and we see a 1 input, we now know there is a 0 followed by a 1. We can go to an accepting state, which we shall call q_1, and which corresponds to condition (1) above. That is, $\delta(q_2, 1) = q_1$.

Finally, we must design the transitions for state q_1. In this state, we have already seen a 01 sequence, so regardless of what happens, we shall still be in a situation where we've seen 01. That is, $\delta(q_1, 0) = \delta(q_1, 1) = q_1$.

Thus, $Q = \{q_0, q_1, q_2\}$. As we said, q_0 is the start state, and the only accepting state is q_1; that is, $F = \{q_1\}$. The complete specification of the automaton A that accepts the language L of strings that have a 01 substring, is

$$A = (\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_1\})$$

where δ is the transition function described above. □

2.2.3 Simpler Notations for DFA's

Specifying a DFA as a five-tuple with a detailed description of the δ transition function is both tedious and hard to read. There are two preferred notations for describing automata:

1. A *transition diagram*, which is a graph such as the ones we saw in Section 2.1.

2. A *transition table*, which is a tabular listing of the δ function, which by implication tells us the set of states and the input alphabet.

Transition Diagrams

A *transition diagram* for a DFA $A = (Q, \Sigma, \delta, q_0, F)$ is a graph defined as follows:

a) For each state in Q there is a node.

b) For each state q in Q and each input symbol a in Σ, let $\delta(q, a) = p$. Then the transition diagram has an arc from node q to node p, labeled a. If there are several input symbols that cause transitions from q to p, then the transition diagram can have one arc, labeled by the list of these symbols.

c) There is an arrow into the start state q_0, labeled *Start*. This arrow does not originate at any node.

d) Nodes corresponding to accepting states (those in F) are marked by a double circle. States not in F have a single circle.

Example 2.2: Figure 2.4 shows the transition diagram for the DFA that we designed in Example 2.1. We see in that diagram the three nodes that correspond to the three states. There is a *Start* arrow entering the start state, q_0, and the one accepting state, q_1, is represented by a double circle. Out of each state is one arc labeled 0 and one arc labeled 1 (although the two arcs are combined into one with a double label in the case of q_1). The arcs each correspond to one of the δ facts developed in Example 2.1. □

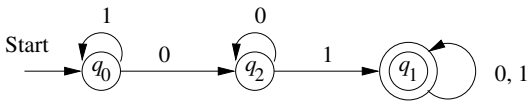


Figure 2.4: The transition diagram for the DFA accepting all strings with a substring 01

Transition Tables

A *transition table* is a conventional, tabular representation of a function like δ that takes two arguments and returns a value. The rows of the table correspond to the states, and the columns correspond to the inputs. The entry for the row corresponding to state q and the column corresponding to input a is the state $\delta(q, a)$.

Example 2.3: The transition table corresponding to the function δ of Example 2.1 is shown in Fig. 2.5. We have also shown two other features of a transition table. The start state is marked with an arrow, and the accepting states are marked with a star. Since we can deduce the sets of states and input symbols by looking at the row and column heads, we can now read from

the transition table all the information we need to specify the finite automaton uniquely. □

	0	1
$\rightarrow q_0$	q_2	q_0
$*q_1$	q_1	q_1
q_2	q_2	q_1

Figure 2.5: Transition table for the DFA of Example 2.1

2.2.4 Extending the Transition Function to Strings

We have explained informally that the DFA defines a language: the set of all strings that result in a sequence of state transitions from the start state to an accepting state. In terms of the transition diagram, the language of a DFA is the set of labels along all the paths that lead from the start state to any accepting state.

Now, we need to make the notion of the language of a DFA precise. To do so, we define an *extended transition function* that describes what happens when we start in any state and follow any sequence of inputs. If δ is our transition function, then the extended transition function constructed from δ will be called $\hat{\delta}$. The extended transition function is a function that takes a state q and a string w and returns a state p — the state that the automaton reaches when starting in state q and processing the sequence of inputs w. We define $\hat{\delta}$ by induction on the length of the input string, as follows:

BASIS: $\hat{\delta}(q, \epsilon) = q$. That is, if we are in state q and read no inputs, then we are still in state q.

INDUCTION: Suppose w is a string of the form xa; that is, a is the last symbol of w, and x is the string consisting of all but the last symbol.[3] For example, $w = 1101$ is broken into $x = 110$ and $a = 1$. Then

$$\hat{\delta}(q, w) = \delta\big(\hat{\delta}(q, x), a\big) \tag{2.1}$$

Now (2.1) may seem like a lot to take in, but the idea is simple. To compute $\hat{\delta}(q, w)$, first compute $\hat{\delta}(q, x)$, the state that the automaton is in after processing all but the last symbol of w. Suppose this state is p; that is, $\hat{\delta}(q, x) = p$. Then $\hat{\delta}(q, w)$ is what we get by making a transition from state p on input a, the last symbol of w. That is, $\hat{\delta}(q, w) = \delta(p, a)$.

[3] Recall our convention that letters at the beginning of the alphabet are symbols, and those near the end of the alphabet are strings. We need that convention to make sense of the phrase "of the form xa."

Example 2.4: Let us design a DFA to accept the language

$$L = \{w \mid w \text{ has both an even number of 0's and an even number of 1's}\}$$

It should not be surprising that the job of the states of this DFA is to count both the number of 0's and the number of 1's, but count them modulo 2. That is, the state is used to remember whether the number of 0's seen so far is even or odd, and also to remember whether the number of 1's seen so far is even or odd. There are thus four states, which can be given the following interpretations:

q_0: Both the number of 0's seen so far and the number of 1's seen so far are even.

q_1: The number of 0's seen so far is even, but the number of 1's seen so far is odd.

q_2: The number of 1's seen so far is even, but the number of 0's seen so far is odd.

q_3: Both the number of 0's seen so far and the number of 1's seen so far are odd.

State q_0 is both the start state and the lone accepting state. It is the start state, because before reading any inputs, the numbers of 0's and 1's seen so far are both zero, and zero is even. It is the only accepting state, because it describes exactly the condition for a sequence of 0's and 1's to be in language L.

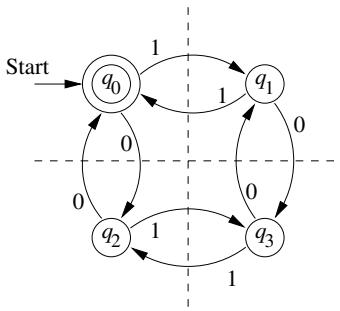


Figure 2.6: Transition diagram for the DFA of Example 2.4

We now know almost how to specify the DFA for language L. It is

$$A = (\{q_0, q_1, q_2, q_3\}, \{0, 1\}, \delta, q_0, \{q_0\})$$

where the transition function δ is described by the transition diagram of Fig. 2.6. Notice how each input 0 causes the state to cross the horizontal, dashed line. Thus, after seeing an even number of 0's we are always above the line, in state

q_0 or q_1 while after seeing an odd number of 0's we are always below the line, in state q_2 or q_3. Likewise, every 1 causes the state to cross the vertical, dashed line. Thus, after seeing an even number of 1's, we are always to the left, in state q_0 or q_2, while after seeing an odd number of 1's we are to the right, in state q_1 or q_3. These observations are an informal proof that the four states have the interpretations attributed to them. However, one could prove the correctness of our claims about the states formally, by a mutual induction in the spirit of Example 1.23.

We can also represent this DFA by a transition table. Figure 2.7 shows this table. However, we are not just concerned with the design of this DFA; we want to use it to illustrate the construction of $\hat{\delta}$ from its transition function δ. Suppose the input is 110101. Since this string has even numbers of 0's and 1's both, we expect it is in the language. Thus, we expect that $\hat{\delta}(q_0, 110101) = q_0$, since q_0 is the only accepting state. Let us now verify that claim.

	0	1
$* \rightarrow q_0$	q_2	q_1
q_1	q_3	q_0
q_2	q_0	q_3
q_3	q_1	q_2

Figure 2.7: Transition table for the DFA of Example 2.4

The check involves computing $\hat{\delta}(q_0, w)$ for each prefix w of 110101, starting at ϵ and going in increasing size. The summary of this calculation is:

- $\hat{\delta}(q_0, \epsilon) = q_0$.
- $\hat{\delta}(q_0, 1) = \delta\big(\hat{\delta}(q_0, \epsilon), 1\big) = \delta(q_0, 1) = q_1$.
- $\hat{\delta}(q_0, 11) = \delta\big(\hat{\delta}(q_0, 1), 1\big) = \delta(q_1, 1) = q_0$.
- $\hat{\delta}(q_0, 110) = \delta\big(\hat{\delta}(q_0, 11), 0\big) = \delta(q_0, 0) = q_2$.
- $\hat{\delta}(q_0, 1101) = \delta\big(\hat{\delta}(q_0, 110), 1\big) = \delta(q_2, 1) = q_3$.
- $\hat{\delta}(q_0, 11010) = \delta\big(\hat{\delta}(q_0, 1101), 0\big) = \delta(q_3, 0) = q_1$.
- $\hat{\delta}(q_0, 110101) = \delta\big(\hat{\delta}(q_0, 11010), 1\big) = \delta(q_1, 1) = q_0$.

□

Standard Notation and Local Variables

After reading this section, you might imagine that our customary notation is required; that is, you *must* use δ for the transition function, use A for the name of a DFA, and so on. We tend to use the same variables to denote the same thing across all examples, because it helps to remind you of the types of variables, much the way a variable i in a program is almost always of integer type. However, we are free to call the components of an automaton, or anything else, anything we wish. Thus, you are free to call a DFA M and its transition function T if you like.

Moreover, you should not be surprised that the same variable means different things in different contexts. For example, the DFA's of Examples 2.1 and 2.4 both were given a transition function called δ. However, the two transition functions are each local variables, belonging only to their examples. These two transition functions are very different and bear no relationship to one another.

2.2.5 The Language of a DFA

Now, we can define the *language* of a DFA $A = (Q, \Sigma, \delta, q_0, F)$. This language is denoted $L(A)$, and is defined by

$$L(A) = \{w \mid \hat{\delta}(q_0, w) \text{ is in } F\}$$

That is, the language of A is the set of strings w that take the start state q_0 to one of the accepting states. If L is $L(A)$ for some DFA A, then we say L is a *regular language*.

Example 2.5: As we mentioned earlier, if A is the DFA of Example 2.1, then $L(A)$ is the set of all strings of 0's and 1's that contain a substring 01. If A is instead the DFA of Example 2.4, then $L(A)$ is the set of all strings of 0's and 1's whose numbers of 0's and 1's are both even. □

2.2.6 Exercises for Section 2.2

Exercise 2.2.1: In Fig. 2.8 is a marble-rolling toy. A marble is dropped at A or B. Levers x_1, x_2, and x_3 cause the marble to fall either to the left or to the right. Whenever a marble encounters a lever, it causes the lever to reverse after the marble passes, so the next marble will take the opposite branch.

***** a) Model this toy by a finite automaton. Let the inputs A and B represent the input into which the marble is dropped. Let acceptance correspond to the marble exiting at D; nonacceptance represents a marble exiting at C.

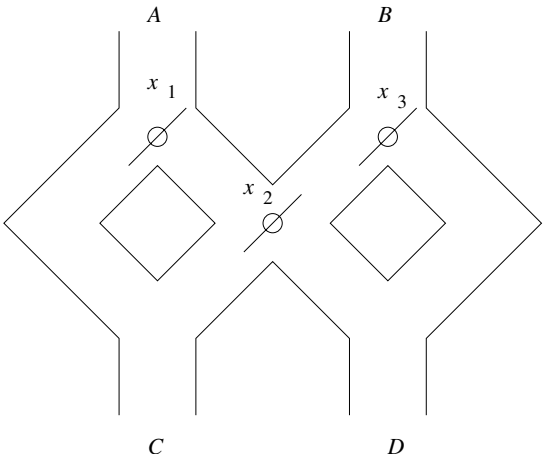


Figure 2.8: A marble-rolling toy

! b) Informally describe the language of the automaton.

c) Suppose that instead the levers switched *before* allowing the marble to pass. How would your answers to parts (a) and (b) change?

*! **Exercise 2.2.2:** We defined $\hat{\delta}$ by breaking the input string into any string followed by a single symbol (in the inductive part, Equation 2.1). However, we informally think of $\hat{\delta}$ as describing what happens along a path with a certain string of labels, and if so, then it should not matter how we break the input string in the definition of $\hat{\delta}$. Show that in fact, $\hat{\delta}(q, xy) = \hat{\delta}\big(\hat{\delta}(q, x), y\big)$ for any state q and strings x and y. *Hint*: Perform an induction on $|y|$.

! **Exercise 2.2.3:** Show that for any state q, string x, and input symbol a, $\hat{\delta}(q, ax) = \hat{\delta}\big(\delta(q, a), x\big)$. *Hint*: Use Exercise 2.2.2.

Exercise 2.2.4: Give DFA's accepting the following languages over the alphabet $\{0, 1\}$:

* a) The set of all strings ending in 00.

b) The set of all strings with three consecutive 0's (not necessarily at the end).

c) The set of strings with 011 as a substring.

! **Exercise 2.2.5:** Give DFA's accepting the following languages over the alphabet $\{0, 1\}$:

a) The set of all strings such that each block of five consecutive symbols contains at least two 0's.

b) The set of all strings whose tenth symbol from the right end is a 1.

c) The set of strings that either begin or end (or both) with 01.

d) The set of strings such that the number of 0's is divisible by five, and the number of 1's is divisible by 3.

!! **Exercise 2.2.6:** Give DFA's accepting the following languages over the alphabet $\{0,1\}$:

* a) The set of all strings beginning with a 1 that, when interpreted as a binary integer, is a multiple of 5. For example, strings 101, 1010, and 1111 are in the language; 0, 100, and 111 are not.

b) The set of all strings that, when interpreted *in reverse* as a binary integer, is divisible by 5. Examples of strings in the language are 0, 10011, 1001100, and 0101.

Exercise 2.2.7: Let A be a DFA and q a particular state of A, such that $\delta(q,a) = q$ for all input symbols a. Show by induction on the length of the input that for all input strings w, $\hat{\delta}(q,w) = q$.

Exercise 2.2.8: Let A be a DFA and a a particular input symbol of A, such that for all states q of A we have $\delta(q,a) = q$.

a) Show by induction on n that for all $n \geq 0$, $\hat{\delta}(q,a^n) = q$, where a^n is the string consisting of n a's.

b) Show that either $\{a\}^* \subseteq L(A)$ or $\{a\}^* \cap L(A) = \emptyset$.

*! **Exercise 2.2.9:** Let $A = (Q, \Sigma, \delta, q_0, \{q_f\})$ be a DFA, and suppose that for all a in Σ we have $\delta(q_0,a) = \delta(q_f,a)$.

a) Show that for all $w \neq \epsilon$ we have $\hat{\delta}(q_0,w) = \hat{\delta}(q_f,w)$.

b) Show that if x is a nonempty string in $L(A)$, then for all $k > 0$, x^k (i.e., x written k times) is also in $L(A)$.

*! **Exercise 2.2.10:** Consider the DFA with the following transition table:

	0	1
$\rightarrow A$	A	B
$*B$	B	A

Informally describe the language accepted by this DFA, and prove by induction on the length of an input string that your description is correct. *Hint*: When setting up the inductive hypothesis, it is wise to make a statement about what inputs get you to each state, not just what inputs get you to the accepting state.

! **Exercise 2.2.11 :** Repeat Exercise 2.2.10 for the following transition table:

	0	1
$\rightarrow *A$	B	A
$*B$	C	A
C	C	C

2.3 Nondeterministic Finite Automata

A "nondeterministic" finite automaton (*NFA*) has the power to be in several states at once. This ability is often expressed as an ability to "guess" something about its input. For instance, when the automaton is used to search for certain sequences of characters (e.g., keywords) in a long text string, it is helpful to "guess" that we are at the beginning of one of those strings and use a sequence of states to do nothing but check that the string appears, character by character. We shall see an example of this type of application in Section 2.4.

Before examining applications, we need to define nondeterministic finite automata and show that each one accepts a language that is also accepted by some DFA. That is, the NFA's accept exactly the regular languages, just as DFA's do. However, there are reasons to think about NFA's. They are often more succinct and easier to design than DFA's. Moreover, while we can always convert an NFA to a DFA, the latter may have exponentially more states than the NFA; fortunately, cases of this type are rare.

2.3.1 An Informal View of Nondeterministic Finite Automata

Like the DFA, an NFA has a finite set of states, a finite set of input symbols, one start state and a set of accepting states. It also has a transition function, which we shall commonly call δ. The difference between the DFA and the NFA is in the type of δ. For the NFA, δ is a function that takes a state and input symbol as arguments (like the DFA's transition function), but returns a set of zero, one, or more states (rather than returning exactly one state, as the DFA must). We shall start with an example of an NFA, and then make the definitions precise.

Example 2.6 : Figure 2.9 shows a nondeterministic finite automaton, whose job is to accept all and only the strings of 0's and 1's that end in 01. State q_0 is the start state, and we can think of the automaton as being in state q_0 (perhaps among other states) whenever it has not yet "guessed" that the final 01 has begun. It is always possible that the next symbol does not begin the final 01, even if that symbol is 0. Thus, state q_0 may transition to itself on both 0 and 1.

However, if the next symbol is 0, this NFA also guesses that the final 01 has begun. An arc labeled 0 thus leads from q_0 to state q_1. Notice that there are

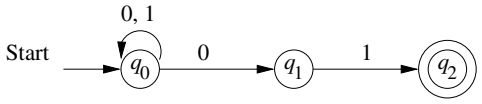


Figure 2.9: An NFA accepting all strings that end in 01

two arcs labeled 0 out of q_0. The NFA has the option of going either to q_0 or to q_1, and in fact it does both, as we shall see when we make the definitions precise. In state q_1, the NFA checks that the next symbol is 1, and if so, it goes to state q_2 and accepts.

Notice that there is no arc out of q_1 labeled 0, and there are no arcs at all out of q_2. In these situations, the thread of the NFA's existence corresponding to those states simply "dies," although other threads may continue to exist. While a DFA has exactly one arc out of each state for each input symbol, an NFA has no such constraint; we have seen in Fig. 2.9 cases where the number of arcs is zero, one, and two, for example.

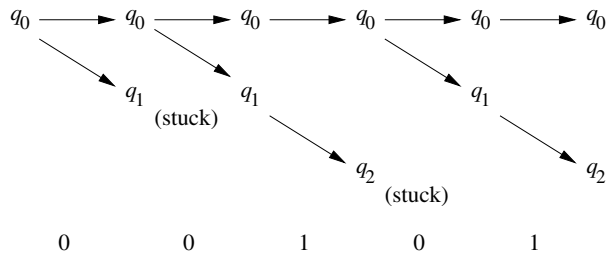


Figure 2.10: The states an NFA is in during the processing of input sequence 00101

Figure 2.10 suggests how an NFA processes inputs. We have shown what happens when the automaton of Fig. 2.9 receives the input sequence 00101. It starts in only its start state, q_0. When the first 0 is read, the NFA may go to either state q_0 or state q_1, so it does both. These two threads are suggested by the second column in Fig. 2.10.

Then, the second 0 is read. State q_0 may again go to both q_0 and q_1. However, state q_1 has no transition on 0, so it "dies." When the third input, a 1, occurs, we must consider transitions from both q_0 and q_1. We find that q_0 goes only to q_0 on 1, while q_1 goes only to q_2. Thus, after reading 001, the NFA is in states q_0 and q_2. Since q_2 is an accepting state, the NFA accepts 001.

However, the input is not finished. The fourth input, a 0, causes q_2's thread to die, while q_0 goes to both q_0 and q_1. The last input, a 1, sends q_0 to q_0 and q_1 to q_2. Since we are again in an accepting state, 00101 is accepted. □

2.3.2 Definition of Nondeterministic Finite Automata

Now, let us introduce the formal notions associated with nondeterministic finite automata. The differences between DFA's and NFA's will be pointed out as we do. An NFA is represented essentially like a DFA:

$$A = (Q, \Sigma, \delta, q_0, F)$$

where:

1. Q is a finite set of *states*.
2. Σ is a finite set of *input symbols*.
3. q_0, a member of Q, is the *start state*.
4. F, a subset of Q, is the set of *final* (or *accepting*) states.
5. δ, the *transition function* is a function that takes a state in Q and an input symbol in Σ as arguments and returns a subset of Q. Notice that the only difference between an NFA and a DFA is in the type of value that δ returns: a set of states in the case of an NFA and a single state in the case of a DFA.

Example 2.7: The NFA of Fig. 2.9 can be specified formally as

$$(\{q_0, q_1, q_2\}, \{0, 1\}, \delta, q_0, \{q_2\})$$

where the transition function δ is given by the transition table of Fig. 2.11. □

	0	1
$\rightarrow q_0$	$\{q_0, q_1\}$	$\{q_0\}$
q_1	$\emptyset$	$\{q_2\}$
$*q_2$	$\emptyset$	$\emptyset$

Figure 2.11: Transition table for an NFA that accepts all strings ending in 01

Notice that transition tables can be used to specify the transition function for an NFA as well as for a DFA. The only difference is that each entry in the table for the NFA is a set, even if the set is a *singleton* (has one member). Also notice that when there is no transition at all from a given state on a given input symbol, the proper entry is $\emptyset$, the empty set.

2.3.3 The Extended Transition Function

As for DFA's, we need to extend the transition function δ of an NFA to a function $\hat{\delta}$ that takes a state q and a string of input symbols w, and returns the set of states that the NFA is in if it starts in state q and processes the string w. The idea was suggested by Fig. 2.10; in essence $\hat{\delta}(q, w)$ is the column of states found after reading w, if q is the lone state in the first column. For instance, Fig. 2.10 suggests that $\hat{\delta}(q_0, 001) = \{q_0, q_2\}$. Formally, we define $\hat{\delta}$ for an NFA's transition function δ by:

BASIS: $\hat{\delta}(q, \epsilon) = \{q\}$. That is, without reading any input symbols, we are only in the state we began in.

INDUCTION: Suppose w is of the form $w = xa$, where a is the final symbol of w and x is the rest of w. Also suppose that $\hat{\delta}(q, x) = \{p_1, p_2, \ldots, p_k\}$. Let

$$\bigcup_{i=1}^{k} \delta(p_i, a) = \{r_1, r_2, \ldots, r_m\}$$

Then $\hat{\delta}(q, w) = \{r_1, r_2, \ldots, r_m\}$. Less formally, we compute $\hat{\delta}(q, w)$ by first computing $\hat{\delta}(q, x)$, and by then following any transition from any of these states that is labeled a.

Example 2.8 : Let us use $\hat{\delta}$ to describe the processing of input 00101 by the NFA of Fig. 2.9. A summary of the steps is:

1. $\hat{\delta}(q_0, \epsilon) = \{q_0\}$.
2. $\hat{\delta}(q_0, 0) = \delta(q_0, 0) = \{q_0, q_1\}$.
3. $\hat{\delta}(q_0, 00) = \delta(q_0, 0) \cup \delta(q_1, 0) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$.
4. $\hat{\delta}(q_0, 001) = \delta(q_0, 1) \cup \delta(q_1, 1) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$.
5. $\hat{\delta}(q_0, 0010) = \delta(q_0, 0) \cup \delta(q_2, 0) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\}$.
6. $\hat{\delta}(q_0, 00101) = \delta(q_0, 1) \cup \delta(q_1, 1) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$.

Line (1) is the basis rule. We obtain line (2) by applying δ to the lone state, q_0, that is in the previous set, and get $\{q_0, q_1\}$ as a result. Line (3) is obtained by taking the union over the two states in the previous set of what we get when we apply δ to them with input 0. That is, $\delta(q_0, 0) = \{q_0, q_1\}$, while $\delta(q_1, 0) = \emptyset$. For line (4), we take the union of $\delta(q_0, 1) = \{q_0\}$ and $\delta(q_1, 1) = \{q_2\}$. Lines (5) and (6) are similar to lines (3) and (4). $\square$

2.3.4 The Language of an NFA

As we have suggested, an NFA accepts a string w if it is possible to make any sequence of choices of next state, while reading the characters of w, and go from the start state to any accepting state. The fact that other choices using the input symbols of w lead to a nonaccepting state, or do not lead to any state at all (i.e., the sequence of states "dies"), does not prevent w from being accepted by the NFA as a whole. Formally, if $A = (Q, \Sigma, \delta, q_0, F)$ is an NFA, then

$$L(A) = \{w \mid \hat{\delta}(q_0, w) \cap F \neq \emptyset\}$$

That is, $L(A)$ is the set of strings w in Σ^* such that $\hat{\delta}(q_0, w)$ contains at least one accepting state.

Example 2.9 : As an example, let us prove formally that the NFA of Fig. 2.9 accepts the language $L = \{w \mid w \text{ ends in } 01\}$. The proof is a mutual induction of the following three statements that characterize the three states:

1. $\hat{\delta}(q_0, w)$ contains q_0 for every w.
2. $\hat{\delta}(q_0, w)$ contains q_1 if and only if w ends in 0.
3. $\hat{\delta}(q_0, w)$ contains q_2 if and only if w ends in 01.

To prove these statements, we need to consider how A can reach each state; i.e., what was the last input symbol, and in what state was A just before reading that symbol?

Since the language of this automaton is the set of strings w such that $\hat{\delta}(q_0, w)$ contains q_2 (because q_2 is the only accepting state), the proof of these three statements, in particular the proof of (3), guarantees that the language of this NFA is the set of strings ending in 01. The proof of the theorem is an induction on $|w|$, the length of w, starting with length 0.

BASIS: If $|w| = 0$, then $w = \epsilon$. Statement (1) says that $\hat{\delta}(q_0, \epsilon)$ contains q_0, which it does by the basis part of the definition of $\hat{\delta}$. For statement (2), we know that ϵ does not end in 0, and we also know that $\hat{\delta}(q_0, \epsilon)$ does not contain q_1, again by the basis part of the definition of $\hat{\delta}$. Thus, the hypotheses of both directions of the if-and-only-if statement are false, and therefore both directions of the statement are true. The proof of statement (3) for $w = \epsilon$ is essentially the same as the above proof for statement (2).

INDUCTION: Assume that $w = xa$, where a is a symbol, either 0 or 1. We may assume statements (1) through (3) hold for x, and we need to prove them for w. That is, we assume $|w| = n + 1$, so $|x| = n$. We assume the inductive hypothesis for n and prove it for $n + 1$.

1. We know that $\hat{\delta}(q_0, x)$ contains q_0. Since there are transitions on both 0 and 1 from q_0 to itself, it follows that $\hat{\delta}(q_0, w)$ also contains q_0, so statement (1) is proved for w.

2. (If) Assume that w ends in 0; i.e., $a = 0$. By statement (1) applied to x, we know that $\hat{\delta}(q_0, x)$ contains q_0. Since there is a transition from q_0 to q_1 on input 0, we conclude that $\hat{\delta}(q_0, w)$ contains q_1.

   (Only-if) Suppose $\hat{\delta}(q_0, w)$ contains q_1. If we look at the diagram of Fig. 2.9, we see that the only way to get into state q_1 is if the input sequence w is of the form $x0$. That is enough to prove the "only-if" portion of statement (2).

3. (If) Assume that w ends in 01. Then if $w = xa$, we know that $a = 1$ and x ends in 0. By statement (2) applied to x, we know that $\hat{\delta}(q_0, x)$ contains q_1. Since there is a transition from q_1 to q_2 on input 1, we conclude that $\hat{\delta}(q_0, w)$ contains q_2.

   (Only-if) Suppose $\hat{\delta}(q_0, w)$ contains q_2. Looking at the diagram of Fig. 2.9, we discover that the only way to get to state q_2 is for w to be of the form $x1$, where $\hat{\delta}(q_0, x)$ contains q_1. By statement (2) applied to x, we know that x ends in 0. Thus, w ends in 01, and we have proved statement (3).

□

2.3.5 Equivalence of Deterministic and Nondeterministic Finite Automata

Although there are many languages for which an NFA is easier to construct than a DFA, such as the language (Example 2.6) of strings that end in 01, it is a surprising fact that every language that can be described by some NFA can also be described by some DFA. Moreover, the DFA in practice has about as many states as the NFA, although it often has more transitions. In the worst case, however, the smallest DFA can have 2^n states while the smallest NFA for the same language has only n states.

The proof that DFA's can do whatever NFA's can do involves an important "construction" called the *subset construction* because it involves constructing all subsets of the set of states of the NFA. In general, many proofs about automata involve constructing one automaton from another. It is important for us to observe the subset construction as an example of how one formally describes one automaton in terms of the states and transitions of another, without knowing the specifics of the latter automaton.

The subset construction starts from an NFA $N = (Q_N, \Sigma, \delta_N, q_0, F_N)$. Its goal is the description of a DFA $D = (Q_D, \Sigma, \delta_D, \{q_0\}, F_D)$ such that $L(D) = L(N)$. Notice that the input alphabets of the two automata are the same, and the start state of D is the set containing only the start state of N. The other components of D are constructed as follows.

- Q_D is the set of subsets of Q_N; i.e., Q_D is the *power set* of Q_N. Note that if Q_N has n states, then Q_D will have 2^n states. Often, not all these states are accessible from the start state of Q_D. Inaccessible states can